

Unidad 3

Programación de comunicaciones

3.1 Conceptos básicos

Programación de servicios y procesos

Curso 2025/2026

Contenido

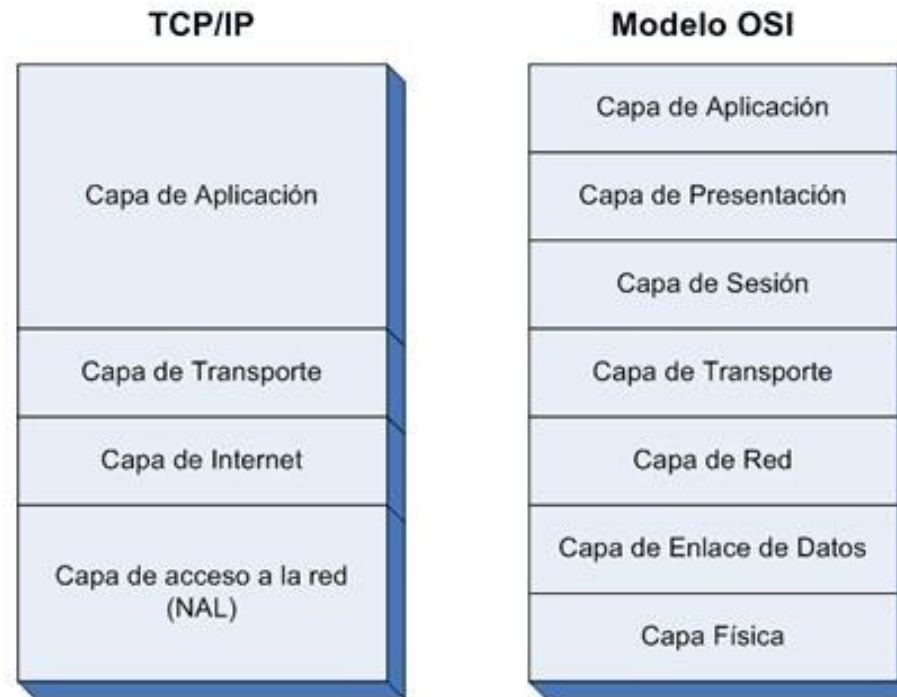
- ▶ Arquitectura TCP/IP
- ▶ Direcciones y nombres
- ▶ Puertos
- ▶ Modelo Cliente / Servidor
- ▶ Sockets

Arquitectura TCP/IP

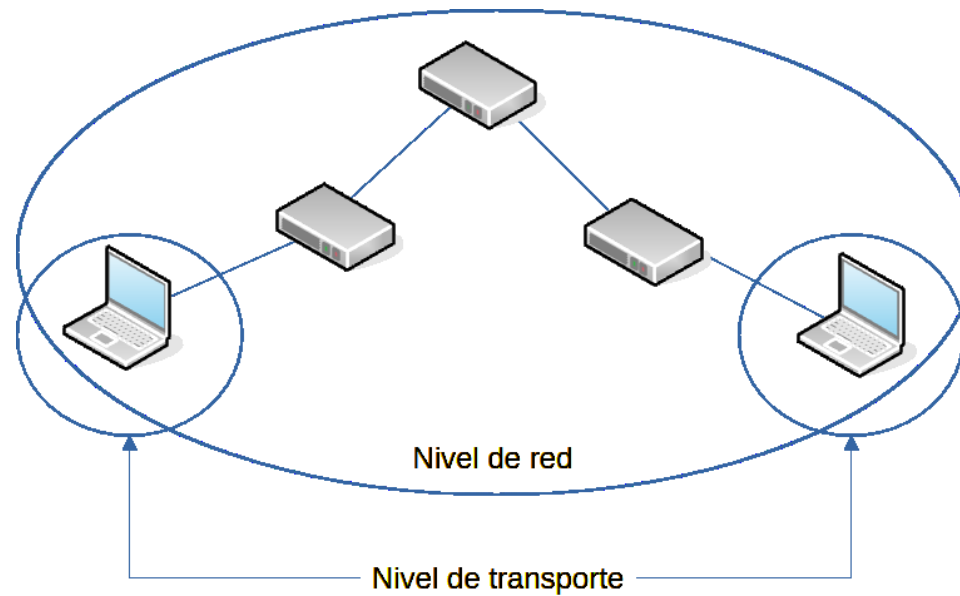
- ▶ La arquitectura de Internet
- ▶ Definida inicialmente para ARPANET
 - ▶ Vinton Cerf y Robert Kahn para DARPA
- ▶ Empezó a funcionar el 1 de enero de 1983
- ▶ Asumida por la IETF en 1986
 - ▶ RFC 791 - IPv4
 - ▶ RFC 793 - TCP

Capas o niveles

- ▶ Capa de acceso a la red: adaptación al medio de transmisión
 - ▶ Fuera del ámbito de IETF
- ▶ Capa de internet (o Red): comunicación a través de la red
- ▶ Capa de transporte: comunicación entre extremos
- ▶ Capa de aplicación: interfaz para aplicaciones



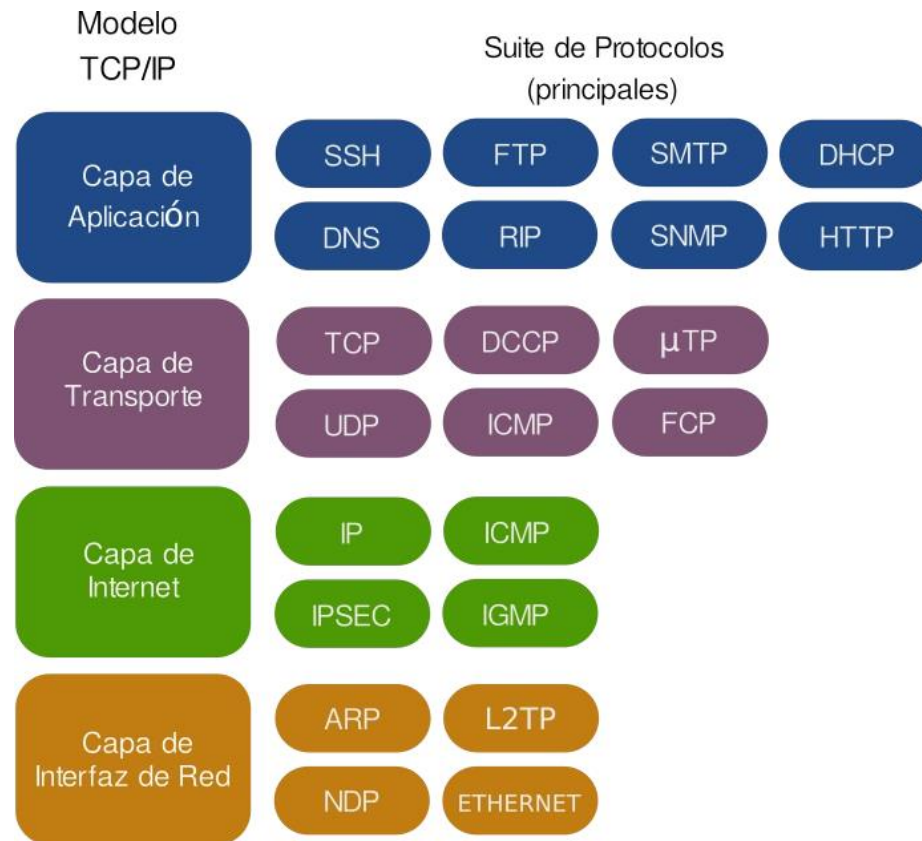
Capas de red y transporte



Protocolos

► Nos centraremos en:

- IP
- TCP
- UDP



Protocolos

▶ IP

- ▶ funciona como una capa de abstracción universal que se encarga de conectar redes heterogéneas (“Everything over IP, IP over everything”)
- ▶ Nos interesa por sus direcciones - identificación de equipos

▶ TCP

- ▶ Orientado a conexión
- ▶ Transmisión fiable
- ▶ Datos divididos y reensamblados en paquetes numerados: **flujos bidireccionales**

▶ UDP

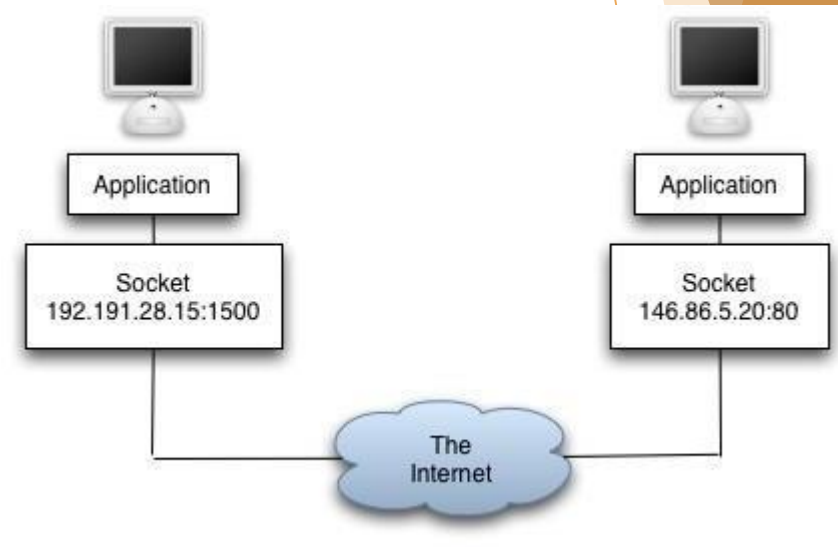
- ▶ No orientado a conexión
- ▶ Transmisión no fiable
- ▶ Envío de **mensajes** llamados datagramas de tamaño limitado (MTU: 1500 bytes)

Direcciones y nombres

- ▶ Direcciones IP
 - ▶ IPv4: 32 bits - ejemplo 193.147.0.29
 - ▶ IPv6: 128 bits - ejemplo 2001:0db8:85a3:0000:0000:8a2e:0370:7334
- ▶ Sistema de nombres: más manejables
 - ▶ URL: Universal Resource Locator
 - ▶ Ejemplo: <https://portal.edu.gva.es/fpcheste/>
- ▶ Resolución de nombres a direcciones: DNS (Domain Name System)
 - ▶ Servidores organizados de forma jerárquica
 - ▶ Dado un equipo se puede obtener su dirección IP

Puertos

- ▶ Empleados por TCP y UDP
 - ▶ La dirección identifica al **equipo**
 - ▶ El puerto a la **aplicación o protocolo dentro del equipo**
- ▶ Una **conexión** se identifica por la **4-tupla**:
 - ▶ Dirección IP origen
 - ▶ Puerto origen
 - ▶ Dirección IP destino
 - ▶ Puerto destino
- ▶ Clave en los sockets, como veremos

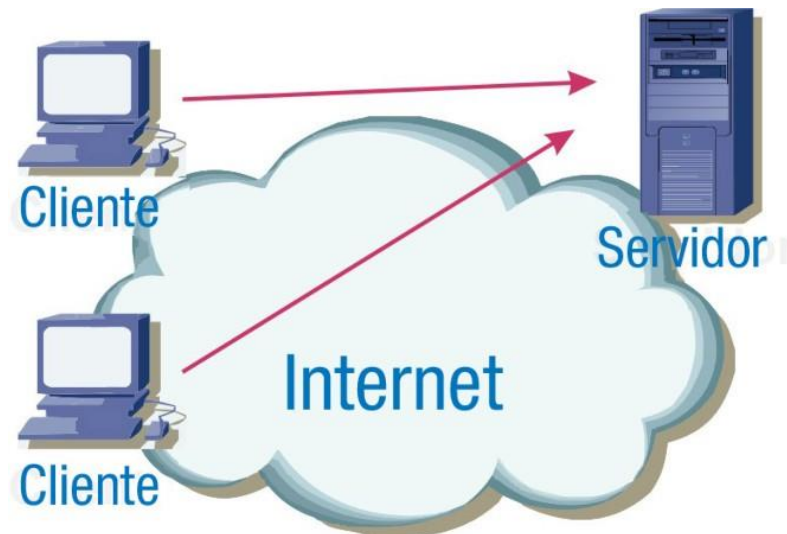


Tipos de puertos

- ▶ Puertos conocidos: 0 - 1023
 - ▶ Reservados a protocolos estándar
 - ▶ Ejemplos: 21 → FTP, 22 → SSH, DNS → 53, HTTP → 80
- ▶ Puertos registrados: 1024 - 49151
 - ▶ Reservados por la IANA para aplicaciones:
 - ▶ Ejemplos: 3306 MySQL, 27017 → MongoDB, 25565 → Minecraft Server
- ▶ Puertos dinámicos o efímeros: 49152 - 65535
 - ▶ Usados por el cliente para conexiones salientes
 - ▶ También para servidores propios sin riesgo de colisiones
 - ▶ Recomendable para pruebas: usar puertos recordables: 50000, 60000...

Modelo cliente - servidor

- ▶ **Servidor:** espera peticiones, recibe datos de entrada y devuelve respuestas
- ▶ **Cliente:** genera peticiones, las envía a un servidor y espera respuestas
- ▶ Idea clave: **el servidor espera, el cliente inicia**

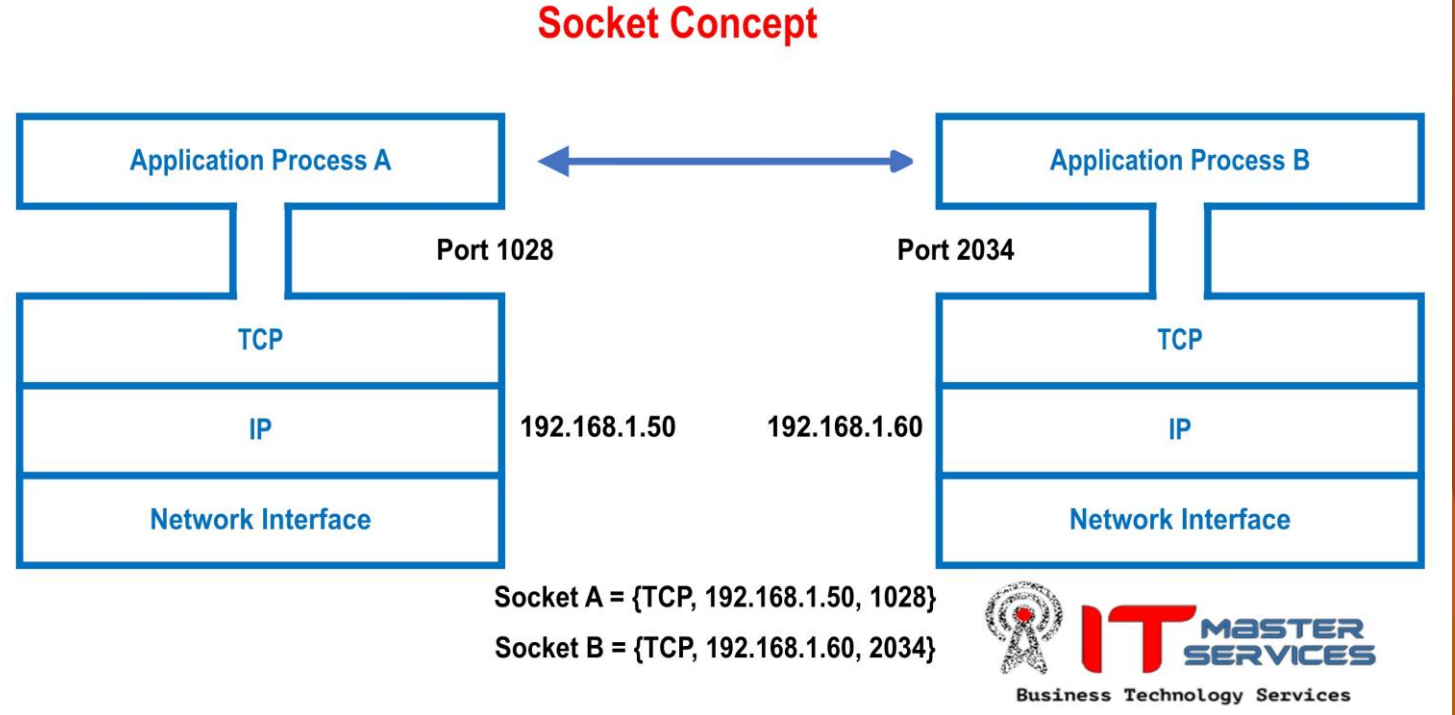


Sockets

- ▶ API *universal* para acceso a TCP y UDP
- ▶ También llamados “Berkley sockets” (originalmente en BSD - Berkeley Software Distribution) - liberada en 1983
- ▶ Adoptadas por POSIX e implementadas en Linux
- ▶ En Windows: **Winsocks**
 - ▶ Diferencias en tratamiento de errores, cierre de conexiones, funciones disponibles
- ▶ La buena noticia: **programando en Java no hay que preocuparse por el S.O.**
 - ▶ las diferencias las gestiona la JVM

Sockets

- ▶ Un socket se identifica por:
 - ▶ Protocolo (TCP o UDP)
 - ▶ dirección IP
 - ▶ puerto
- ▶ Para comunicar dos aplicaciones en máquinas remotas
 - ▶ cada aplicación tiene un su socket abierto
 - ▶ los sockets deben estar conectados
 - ▶ se envían y reciben datos por el socket



Sockets en el mismo puerto

- ▶ Es posible tener varios clientes conectados en el mismo puerto en uno de los extremos
- ▶ Incluso con la misma máquina si el puerto origen es distinto
- ▶ Recordemos que un socket se identifica por la 4-tupla:
 - ▶ dirección origen/destino puerto origen/destino

